

Как писать комментарии в коде

Поддержка кода занимает гораздо больше времени, чем разработка. Поэтому одна из главных целей разработки – уменьшить стоимость поддержки исходного кода. Нет смысла экономить на написании комментариев – это не выгодно. Комментарии (пояснения) к [исходному коду](#) – его неотъемлемая часть. Поясняющие комментарии кода являются его отражением и предназначаются для всех, кто будет заниматься его поддержкой.

[Синтаксис](#) комментариев определяется [языком программирования](#) и [стандартами оформления кода](#), а их содержание несколькими правилами.

Пишите содержательно

Если комментарий отвечает на вопрос «что происходит?», он лишний, т.к. мы получаем дублирование информации в коде и комментарии. Особую опасность представляют случаи, когда при правке исходного кода забывают менять комментарии. В этом случае невозможно без дополнительной отладки понять, что происходит.

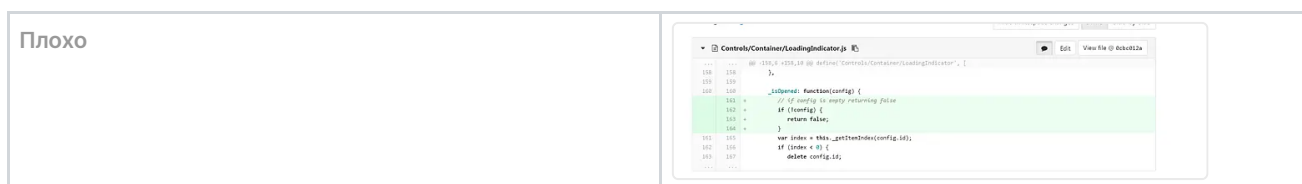


Рисунок 1. Комментарий бесполезный, описывает происходящее

Хорошей практикой программирования является выделение кода в отдельные функции, имена которых заменяют собой комментарии.

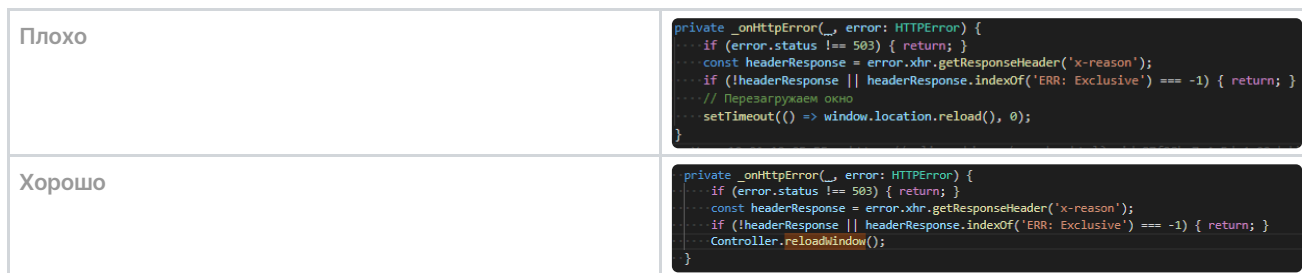


Рисунок 2. Код без выделения функций и с ним (плохо /хорошо)

Иногда из самого кода сложно понять его цель, и почему именно так он написан. Комментарий должен помогать разбираться в причинах, побудивших написать этот код.

Возвращаясь к примеру, показанному на рисунке 2, мы можем сказать, что выделение функции помогло пониманию кода, но комментарий дополнительно поясняет, почему мы используем `setTimeout`.

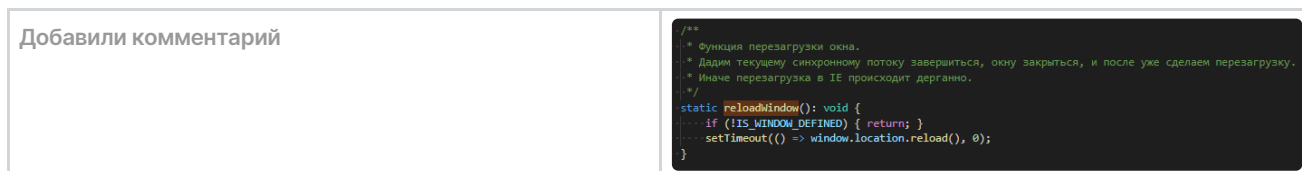


Рисунок 3. Пояснения в комментарии с ответом на вопрос: «почему именно так?»

Хороший комментарий должен отвечать на вопросы «почему именно так?», «зачем это здесь?». Но когда смысл действий сложен для понимания, помимо объяснений «зачем это здесь?» можно добавить комментарий, который подробнее опишет «что здесь происходит».

```
1010 // FIXME: backward compatibility for check via Core/core-instance::instanceOfModule()
1011 Model.prototype['[kS.Data/Entity/Model]'] = true;
1012 // FIXME: backward compatibility for Core/core-extend: Model should have exactly its own property 'produceInstance'
1013 // @ts-ignore
1014 Model.produceInstance = Record.produceInstance;
1015
1016 register('Types/entity:Model', Model, {instantiate: false});
1017 // FIXME: deprecated
1018 register('entity.model', Model);
```

Рисунок 4. Пояснения в комментарии «почему именно так»

Комментарии, которые могут быть полезны

Добавляйте в комментарий краткое описание взаимодействия подсистем

Если ваше решение предполагает наличие каких-либо взаимодействий участков подсистем. Вначале кратко опишите алгоритм работы вашего кода.

```
1  /// <amd-module name="Data/_type/adapter/Sbis" />
2  /**
3   * Адаптер для данных в формате СБИС.
4   * Работает с форматом данных, который использует БЛ СБИС.
5   * Примеры можно посмотреть в модулях {@link WS.Data/Adapter/SbisRecord} и {@link WS.Data/Adapter/SbisTable}.
6   * @class WS.Data/Adapter/Sbis
7   * @extends WS.Data/Adapter/Abstract
8   * @public
9   * @author Мальцев А.А.
10  */
11
12  import Abstract from './Abstract';
13  import SbisTable from './SbisTable';
14  import SbisRecord from './SbisRecord';
15  import FIELD_TYPE from './SbisFieldType';
16  import {di} from '../util';
17
18  export default class Sbis extends Abstract /** @lends WS.Data/Adapter/Sbis.prototype */{
19
20    forTable(data) {
21      return new SbisTable(data);
22    }
23
24    forRecord(data) {
25      return new SbisRecord(data);
26    }
27  }
```

Рисунок 5. Описание взаимодействия подсистем в комментарии

Комментарии / TODO сопровождайте ссылкой на ошибку или задачу

Если вам пришлось решить задачу по причине чьей-то ошибки, так и напишите об этом в комментарии, и укажите ссылку на «Ошибка в разработке». При этом в задаче должно быть требование исправить TODO. Указание ошибки позволит потом вовремя сориентироваться, получить информацию об исправлении ошибки и исправить код.

```

protected _getTransport(url: string, method = "POST", dataType?: string): ITransport {
    if (!this.__transports[method]) {
        this.__transports[method] = new this._$uploadTransport({
            contentType: false,
            dataType: 'json',
            method,
            url
        });
    }
    this.__transports[method].setUrl(url);
    /*
    * TODO как чтобы не хранить отдельный экземпляр транспорта.
    * Поменять, сразу как транспорт перейдет на передачу всех параметров вызова в аргументы, как в Fetch api
    * https://online.sbis.ru/opendoc.html?guid=c2d85f9e-0839-4c08-a64f-4e32a5d90733
    */
    this.__transports[method]._options.dataType = dataType || "json";
    return this.__transports[method];
}

```

Рисунок 6. Комментарий TODO

Описывайте в комментариях проблему, с которой столкнулись, и не смогли решить

Часто в процессе работы мы сталкиваемся с какими-то ограничениями, например, с непониманием, как именно что-то работает. Если вы не понимаете чего-то, не бойтесь – так и укажите в комментарии. Только не наполняйте код такими комментариями как «Ничего не понятно» или «Код неподъемный», т.к. такой комментарий неинформативен.

Лучше уточните, что именно вам не понятно.

В таком случае человек, который встретится с этим кодом в следующий раз, поймет, что нет никаких тайных знаний и исключительных ситуаций, и ему будет легче принять решение, что делать с этим кодом. Он поймет, что именно вам было не понятно, и сможет переписать код.

```

/**
 * Получение объекта WebSocket или создание его с помощью url
 * @param url - url для подключения вебсокета
 * @return Promise<WebSocket>
 */
static getInstance(url: string=null): Promise<WebSocket> {
    if (WebSocketConnect.websocket && (WebSocketConnect.url == url || url === null) &&
        ([WebSocket.CLOSING, WebSocket.CLOSED].indexOf(WebSocketConnect.websocket.readyState) === -1)) {
        return Promise.resolve(WebSocketConnect.websocket);
    }
    if (WebSocketConnect.websocket) {
        WebSocketConnect.websocket.close();
    }
    try {
        let subProtocols = ['stomp'];
        WebSocketConnect.websocket = new SockJSTransport(url, subProtocols);
        return Promise.resolve(WebSocketConnect.websocket);
    } catch (e) {
        // Иногда ff не может корректно создать вебсокета. Подключаемся заново.
        if (e.message.indexOf('NS_ERROR_SOCKET_CREATE_FAILED') > -1) {
            return Promise.resolve(WebSocketConnect.getInstance(url));
        }
        return Promise.reject(new ConnectError(e.message || ERROR_WEBSOCKET_CONNECT));
    }
}

```

Рисунок 7. Комментарий с описанием проблемы

Лишние комментарии

Не пишите комментарии, не имеющие отношение к коду

Помните, что чрезмерные комментарий, не имеющие отношение к исходному коду, а также прочие ресурсы могут привести к замедлению работы с источником исходного кода за счет увеличения объемов.

Избегайте оценок и будьте вежливыми

Не описывайте свои эмоции в комментариях. Особенно, работая с чужим кодом, не стремитесь выразить в комментариях свое возмущение кодом с помощью восклицательных знаков и недоброжелательных фраз. Восклицательный знак несет неуместную эмоциональную окраску,

а негативные комментарии вовсе недопустимы. Пишите комментарии в доброжелательном сдержанном тоне без жаргонных выражений и негативных фраз. Уважайте себя и коллег.